

Intent Understanding

1. Course Content

- Master the ability to customize unique user intent understanding using the RAG knowledge base.

[!IMPORTANT]

- Intent understanding is designed to increase the synergy between the robot and the human, allowing the robot to understand the user more uniquely. This function should not be used to perform "strange" or "illogical" tasks.

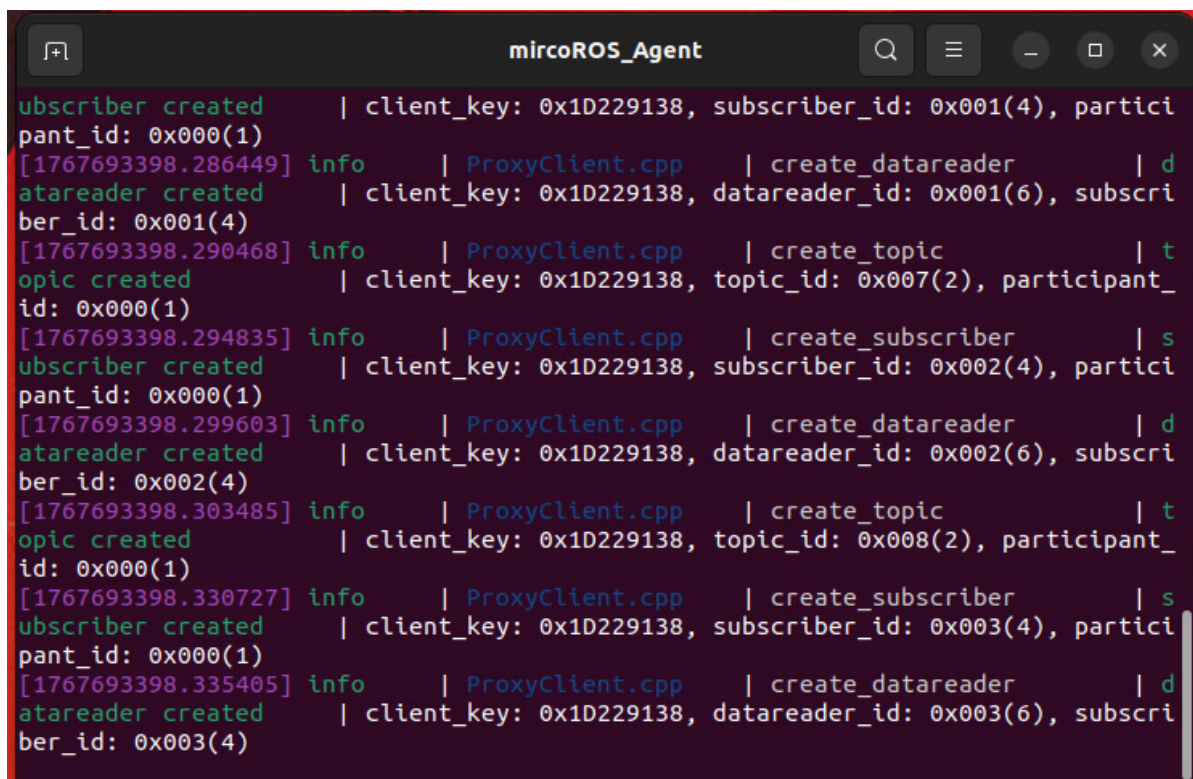
2. Starting the Agent

Note: If it is already started, it does not need to be started again.

Enter the command on the vehicle terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



```
subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1767693398.286449] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1767693398.290468] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_
id: 0x000(1)
[1767693398.294835] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1767693398.299603] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
[1767693398.303485] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x008(2), participant_
id: 0x000(1)
[1767693398.330727] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x1D229138, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1767693398.335405] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
```

2.1 Configuring the Intent Mapping File

- This file is used to store the user's vague intent and the corresponding task operation desired by the robot.
- Open the example file in this tutorial folder. Following the reference format, you can add multiple customized intents. Here's a simple example:

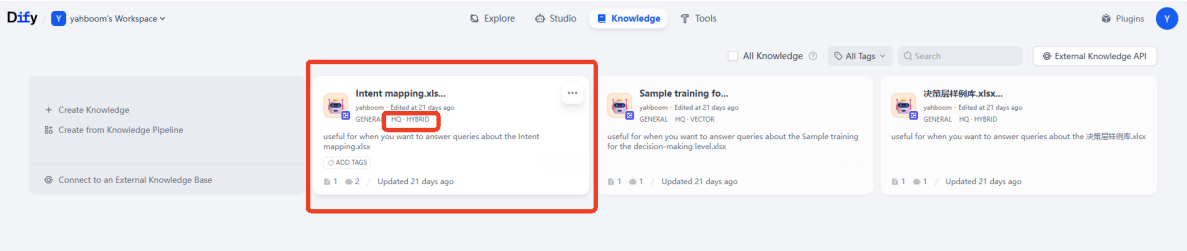
query	answer
I'm thirsty	1. Navigate to the kitchen, 2. Check if there's bottled water or a drink, 3. If a drink is available, return to the starting location and inform the user.

2.2 Configuring the Knowledge Base

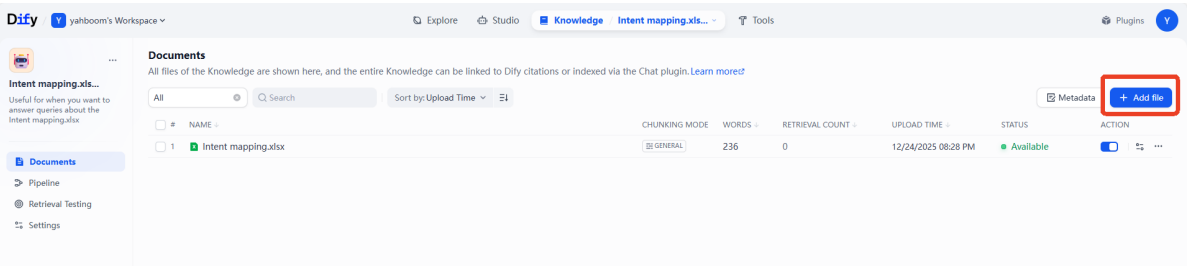
- Next, we need to upload our edited intent mapping file to the `RAG` knowledge base in `Diffy`.

[!NOTE]

- For detailed instructions on operating the RAG knowledge base, please refer to the tutorial in `[04.AI Model Development]–[06.Deploy the RAG knowledge base]`.
- Diffy comes with a default `Intent mapping` reference knowledge base to demonstrate how to use intent understanding features.



- You can modify the `Intent mapping.xlsx` template, or you can delete it and add your own file.



[!TIP]

It's worth noting:

To ensure optimal intent understanding performance, it's recommended to set the `Intent mapping` intent understanding knowledge base to high-quality mode, as intent understanding often requires retrieving relevant fragments between similar semantic cues.

3. Running Examples

3.1 Starting the Program

- On the vehicle's infotainment system, open the terminal and enter the command to start the AI agent function:

```
ros2 launch m3_bringup llm_agent_control.launch.py text_chat_mode:=True
```

- On the vehicle's infotainment system, start the navigation function:

(ORIN & PI5 mainboard)

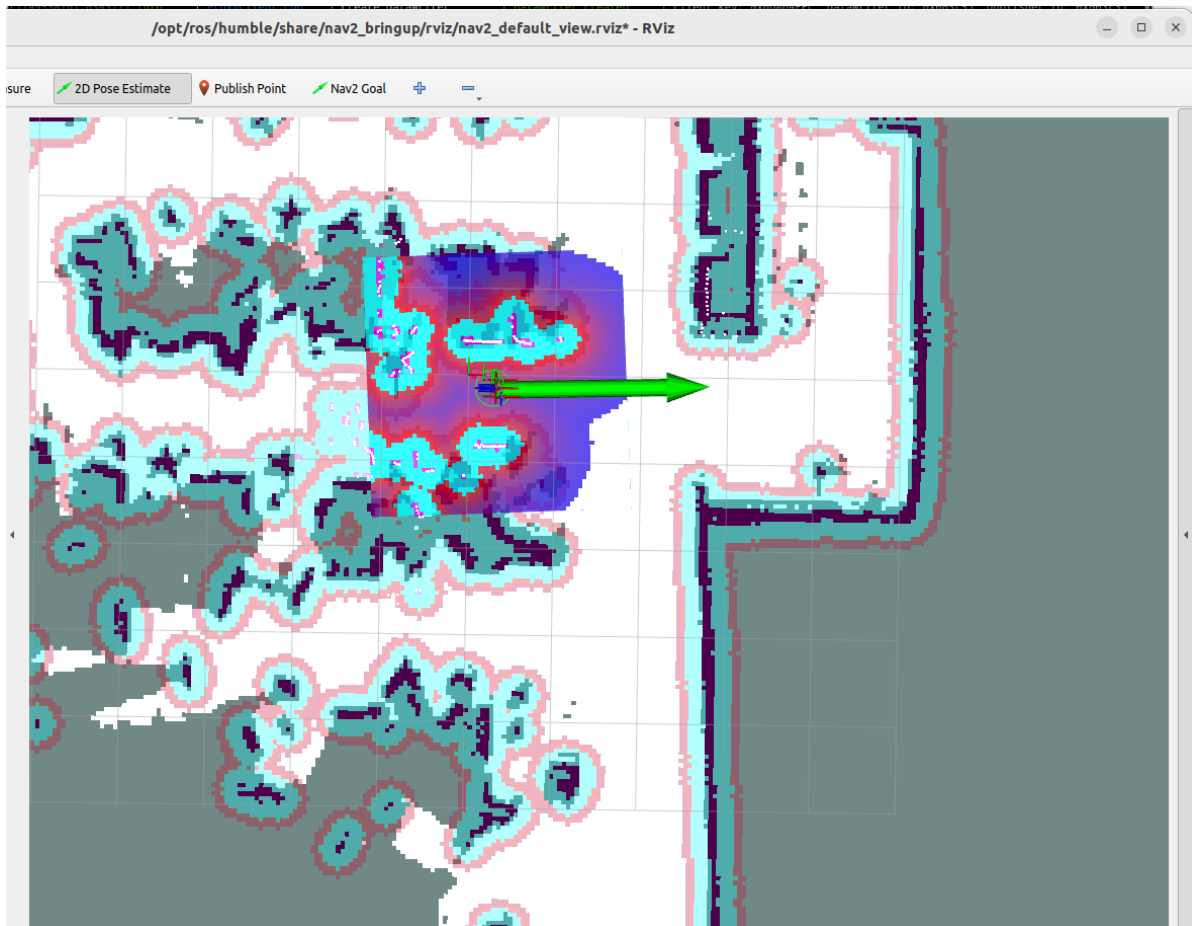
```
ros2 launch m3_bringup nav2.launch.py
```

(RDK mainboard) Step-by-step execution, visualized on the accompanying virtual machine

Rosmaster-M3-Ubuntu22.04 :

```
#Virtual machine side
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
#Host side
ros2 launch m3_bringup nav2.launch.py gui:=False
```

Then, follow the navigation startup process to initialize positioning. This will open the rviz2 visualization interface. Click **2D Pose Estimate** in the top toolbar to enter selection mode. Roughly mark the robot's location and orientation on the map. After initializing positioning, the preparation is complete.



3.2 Test Case

- Running the text interaction node

```
ros2 run m3_common_demo text_chat
```

Here is a reference test case; users can create their own dialogue commands.

- I'm currently in the master bedroom, and I'm feeling a bit thirsty.

Enter the test case in the text-based interactive terminal:

```
jetson@yahboom: ~  
jetson@yahboom: ~ 79x15  
jetson@yahboom:~$ ros2 run text_chat text_chat  
user input: 我现在在主卧室，我感觉有点口渴了
```

The steps for the decision-making layer model planning task are:

```
okay😊, let me think for a moment... \[INFO] [1750496517.715551973] [text_chat_  
node]: robot response: The upcoming task to be carried out: 导航前往厨房区域，观  
察夹取矿泉水，导航到主卧室，放下矿泉水  
user input: \[INFO] [1750496530.302491350] [text_chat_node]: robot response: "act
```

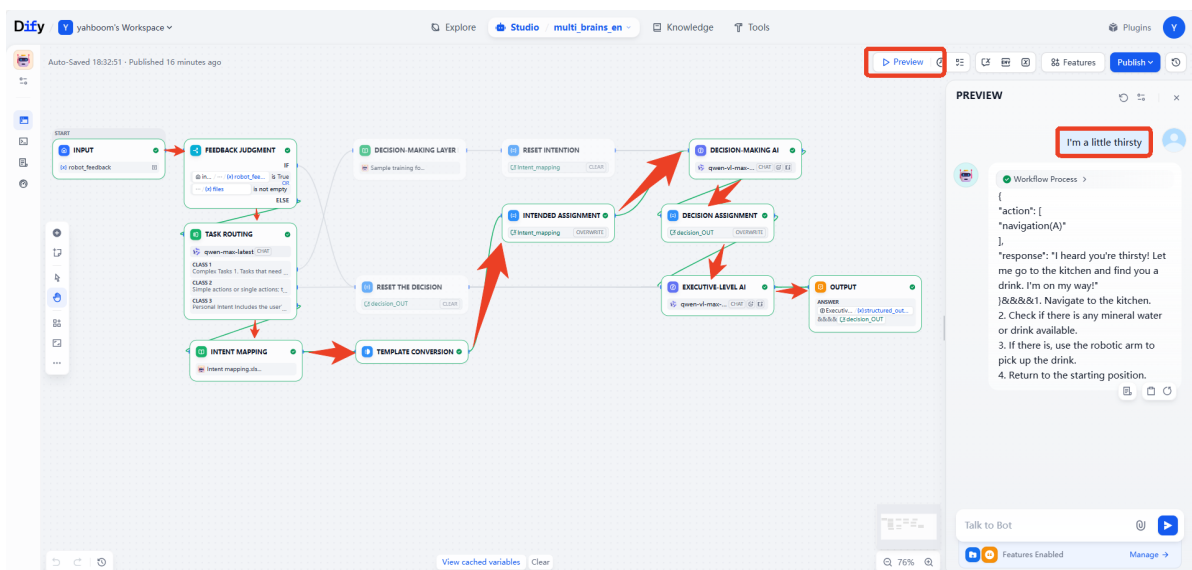
Following the task steps of the decision-making layer large model planning, execute them sequentially:

```
user input: \[INFO] [1750496520.302491350] [text_chat_node]: robot response: "act  
ion": ['navigation(G)'], "response": 好的，我这就去厨房帮你拿矿泉水。就像一个忠  
诚的骑士一样，为你的需求而行动！  
\[INFO] [1750496539.481789151] [text_chat_node]: robot response: "action": ['seew  
hat()'], "response": 我已经到达厨房了，现在开始寻找矿泉水。让我用我的火眼金睛来  
定位它！  
\[INFO] [1750496542.712828693] [text_chat_node]: robot response: "action": ['gras  
p_obj(350,65,450,310)'], "response": 找到了！矿泉水就在那里，我马上把它夹取过来  
。就像魔术师一样，变~
```

4. Effect Debugging

4.1 Visual Workflow

- Open the corresponding language version of the multi_brains application, then click preview. You can input test content to view the data flow.



- You can also open the workflow to view the time taken for each step and the input and output content of each process.

Plugins

Y

Preview

ENV

Features

Publish

PREVIEW

I'm a little thirsty

Workflow Process

INPUT

19.584 ms

FEEDBACK JU...

42.149 ms

TASK...

954 tokens · 2.302 s

INTENT MAPPING

3.919 s

TEMPLATE C...

353.109 ms

INTENDED AS...

28.830 ms

DECI...

737 tokens · 3.185 s

DECISION ASS...

32.704 ms

EX...

2.479K tokens · 3.176 s

OUTPUT

39.530 ms

```
{
  "action": [
    "navigation(A)"
  ],
  "response": "I heard you're thirsty!
Let me go to the kitchen and find
a drink. I'm on my way!"
}
```

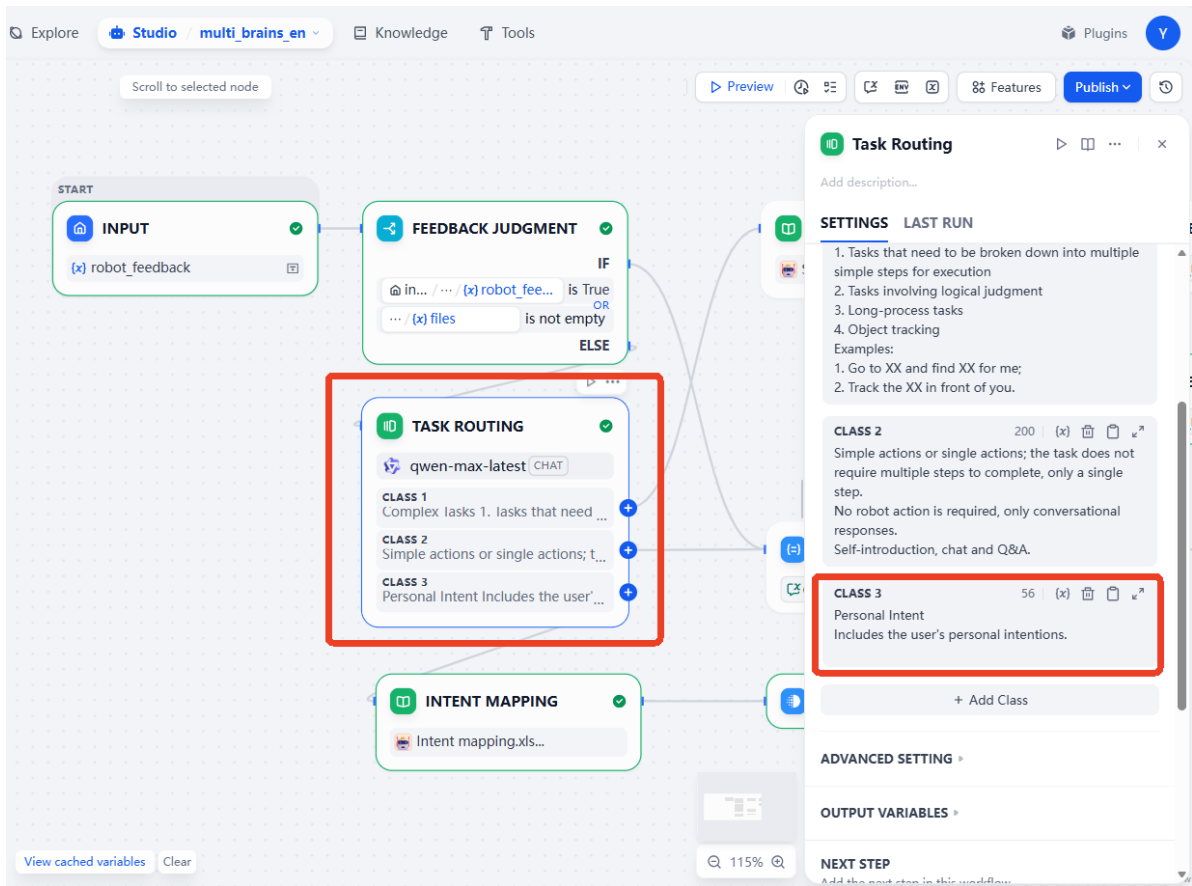
Talk to Bot

a drink. I'm on my way!"

Features Enabled

Manage

- If the task routing module here is inaccurate in certain contexts and is not categorized into `Category 3: Personal Intent`, you can add restrictions on personal intent or supplement the context description here.



- Additionally, the recall effect of the intent understanding statement to be tested in the knowledge base of the intent mapping can also be tested separately.

The screenshot shows the 'Retrieval Test' interface in the Dify Knowledge base. The 'SOURCE TEXT' is 'I'm a little thirsty'. The 'Retrieved Chunks' section shows a single chunk with a score of 0.85. The 'Records' table below shows the test results:

QUERY CONTENT	SOURCE	TIME
I'm a little thirsty	Retrieval Test	01/14/2026 06:35 PM
I'm a little thirsty	App	01/14/2026 06:32 PM